

CARE TRANSITION ANALYTICS - EDA

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")

In [3]: df = pd.read_csv("HMS_Unaccompanied_Allen_Children_Program.csv")

In [4]: df.head()
```

	Date	Children apprehended and placed in CBP custody*	Children in CBP custody	Children transferred out of CBP custody	Children in HHS Care	Children discharged from HMS Care
0	December 21, 2025		6.0	18.0	11.0	2,484
1	December 18, 2025		11.0	50.0	6.0	2,472
2	December 17, 2025		7.0	31.0	11.0	2,481
3	December 16, 2025		8.0	54.0	15.0	2,468
4	December 15, 2025		11.0	42.0	9.0	2,470

```
In [5]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1170 entries, 0 to 1169
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   Date                  720 non-null    object
 1   Children apprehended and placed in CBP custody*  720 non-null    float64
 2   Children in CBP custody  720 non-null    float64
 3   Children transferred out of CBP custody          720 non-null    float64
 4   Children in HHS Care    720 non-null    object
 5   Children discharged from HMS Care               720 non-null    float64
dtypes: float64(4), object(2)
memory usage: 55.8+ KB

In [6]: df.describe()
```

	Children apprehended and placed in CBP custody*	Children in CBP custody	Children transferred out of CBP custody	Children discharged from HMS Care
count	720.000000	720.000000	720.000000	720.000000
mean	93.523611	171.494444	128.668056	173.406944
std	72.646625	126.354965	97.322012	125.702841
min	0.000000	7.000000	0.000000	0.000000
25%	12.000000	36.000000	14.000000	19.750000
50%	99.000000	193.000000	157.000000	181.000000
75%	147.250000	263.250000	199.250000	267.000000
max	333.000000	531.000000	440.000000	505.000000

```
In [7]: df.columns

Out[7]: Index(['Date', 'Children apprehended and placed in CBP custody*', 'Children in CBP custody', 'Children transferred out of CBP custody', 'Children in HHS Care', 'Children discharged from HMS Care'], dtype='object')

In [8]: ## Rename Columns (Clean)

df.columns = [
    "Date",
    "CBP_Apprehended",
    "CBP_In_Custody",
    "CBP_Transferred",
    "HHS_In_Care",
    "HHS_Discharged"
]

In [9]: df.columns

Out[9]: Index(['Date', 'CBP_Apprehended', 'CBP_In_Custody', 'CBP_Transferred', 'HHS_In_Care', 'HHS_Discharged'], dtype='object')

In [10]: ## Cleaning Data

df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')

In [11]: df.head(3)
```

Date	CBP_Apprehended	CBP_In_Custody	CBP_Transferred	HHS_In_Care	HHS_Discharged
719 2023-01-12	33.0	53.0	34.0	6,566	436.0
718 2023-01-22	32.0	49.0	39.0	7,122	227.0
717 2023-01-23	32.0	50.0	39.0	7,280	181.0

```
In [12]: df.isnull().sum()

Out[12]: Date                450
CBP_Apprehended            450
CBP_In_Custody              450
CBP_Transferred             450
HHS_In_Care                 450
HHS_Discharged              450
dtype: int64

In [13]: df.duplicated()

Out[13]: 719    False
718    False
717    False
716    False
715    False
...
1165    True
1166    True
1167    True
1168    True
1169    True
Length: 1170, dtype: bool

In [14]: # Fill missing values

df.fillna(method="ffill", inplace=True)

C:\Users\sahil\AppData\Local\Temp\ipykernel_11548\4842571280.py:3: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() on obj.bfill() instead.
df.fillna(method="ffill", inplace=True)

In [15]: df.isnull().sum()

Out[15]: Date                0
CBP_Apprehended            0
CBP_In_Custody              0
CBP_Transferred             0
HHS_In_Care                 0
HHS_Discharged              0
dtype: int64

In [21]: df.dtypes

Out[21]: Date                datetime64[ns]
CBP_Apprehended            float64
CBP_In_Custody              float64
CBP_Transferred             float64
HHS_In_Care                 object
HHS_Discharged              float64
Transferred_Efficiency      float64
dtype: object

In [22]: cols = [
    "CBP_Apprehended",
    "CBP_In_Custody",
    "CBP_Transferred",
    "HHS_In_Care",
    "HHS_Discharged"
]

for col in cols:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace(",", "")
        .str.strip()
    )

df[col] = pd.to_numeric(df[col], errors="coerce") # convert to numeric

In [23]: df[cols].isnull().sum()

Out[23]: CBP_Apprehended            0
CBP_In_Custody              0
CBP_Transferred             0
HHS_In_Care                 0
HHS_Discharged              0
dtype: int64

In [24]: df[cols] = df[cols].fillna(method="ffill")

C:\Users\sahil\AppData\Local\Temp\ipykernel_11548\2892447748.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() on obj.bfill() instead.
df[cols] = df[cols].fillna(method="ffill")
```

Feature Engineering (KPIs)

```
In [26]: df.columns

Out[26]: Index(['Date', 'CBP_Apprehended', 'CBP_In_Custody', 'CBP_Transferred', 'HHS_In_Care', 'HHS_Discharged', 'Transferred_Efficiency', 'Discharge_Efficiency', 'Throughput', 'Backlog'], dtype='object')

In [27]: #Transfer Efficiency

df['Transferred_Efficiency'] = df['CBP_Transferred'] / df['CBP_In_Custody']

In [29]: #Discharge_Efficiency

df['Discharge_Efficiency'] = df['HHS_Discharged'] / df['HHS_In_Care']

In [30]: # Throughput

df['Throughput'] = df['HHS_Discharged'] / df['CBP_Apprehended']

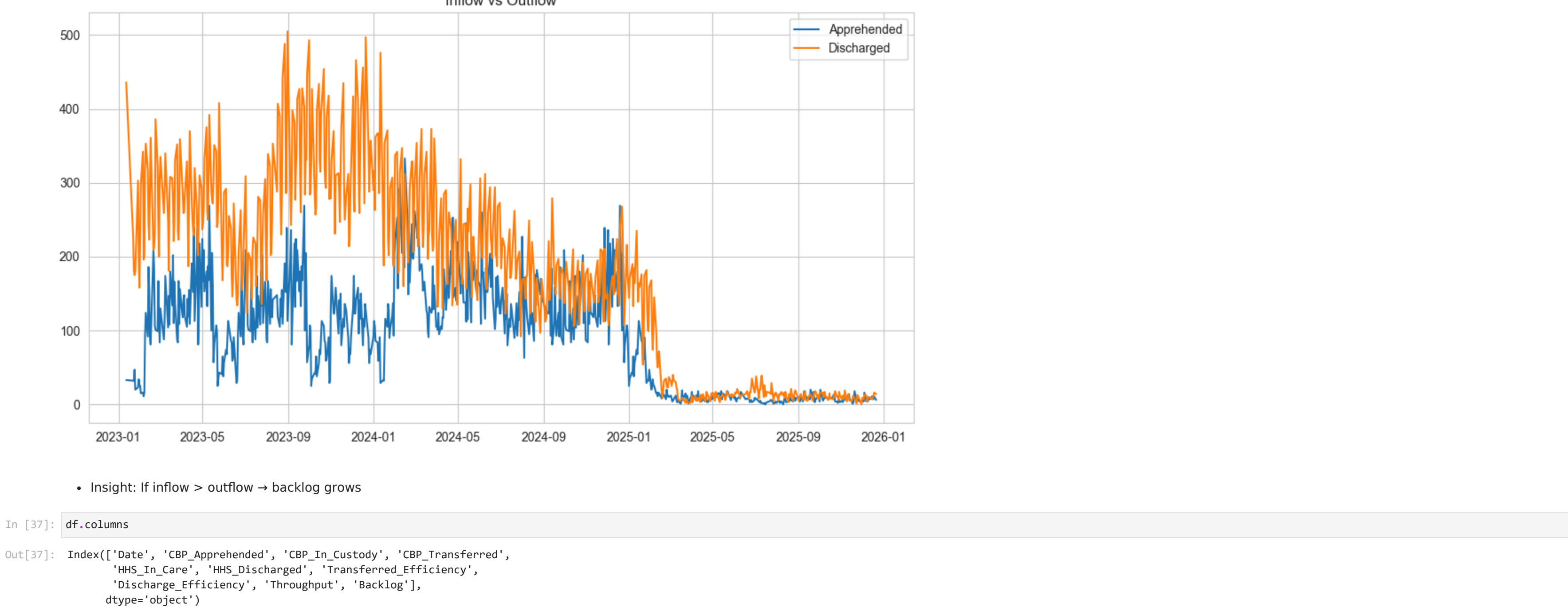
In [32]: # Backlog

df['Backlog'] = (df['CBP_In_Custody'] + df['HHS_In_Care']) - df['HHS_Discharged']

In [33]: df.head()
```

Date	CBP_Apprehended	CBP_In_Custody	CBP_Transferred	HHS_In_Care	HHS_Discharged	Transferred_Efficiency	Discharge_Efficiency	Throughput	Backlog
719 2023-01-12	33.0	53.0	34.0	6566	436.0	0.641509	0.066403	13.212121	6183.0
718 2023-01-22	32.0	49.0	39.0	7122	227.0	0.795918	0.031873	7.093750	6944.0
717 2023-01-23	32.0	50.0	39.0	7280	181.0	0.780000	0.024863	5.656250	7149.0
716 2023-01-24	47.0	42.0	47.0	7433	175.0	1.119048	0.023544	3.723404	7300.0
715 2023-01-25	20.0	22.0	41.0	7538	180.0	1.863636	0.023879	9.000000	7380.0

visualization



• Insight: If inflow > outflow → backlog grows

```
In [37]: df.columns

Out[37]: Index(['Date', 'CBP_Apprehended', 'CBP_In_Custody', 'CBP_Transferred', 'HHS_In_Care', 'HHS_Discharged', 'Transferred_Efficiency', 'Discharge_Efficiency', 'Throughput', 'Backlog'], dtype='object')

In [38]: # Efficiency Trends

df['Transfer_7d'] = df['Transferred_Efficiency'].rolling(7).mean()
df['Discharge_7d'] = df['Discharge_Efficiency'].rolling(7).mean()

plt.figure(figsize=(12,6))
plt.plot(df['date'], df['Transfer_7d'], label='Transferred Efficiency')
plt.plot(df['date'], df['Discharge_7d'], label='Discharge Efficiency')
plt.legend()
plt.title("Efficiency Trends (7-Day Avg)")
plt.show()
```



```
In [42]: # Bottleneck Detection

threshold = df['Backlog'].quantile(0.9)
bottleneck = df[df['Backlog'] > threshold]
bottleneck[['Date', 'Backlog']]

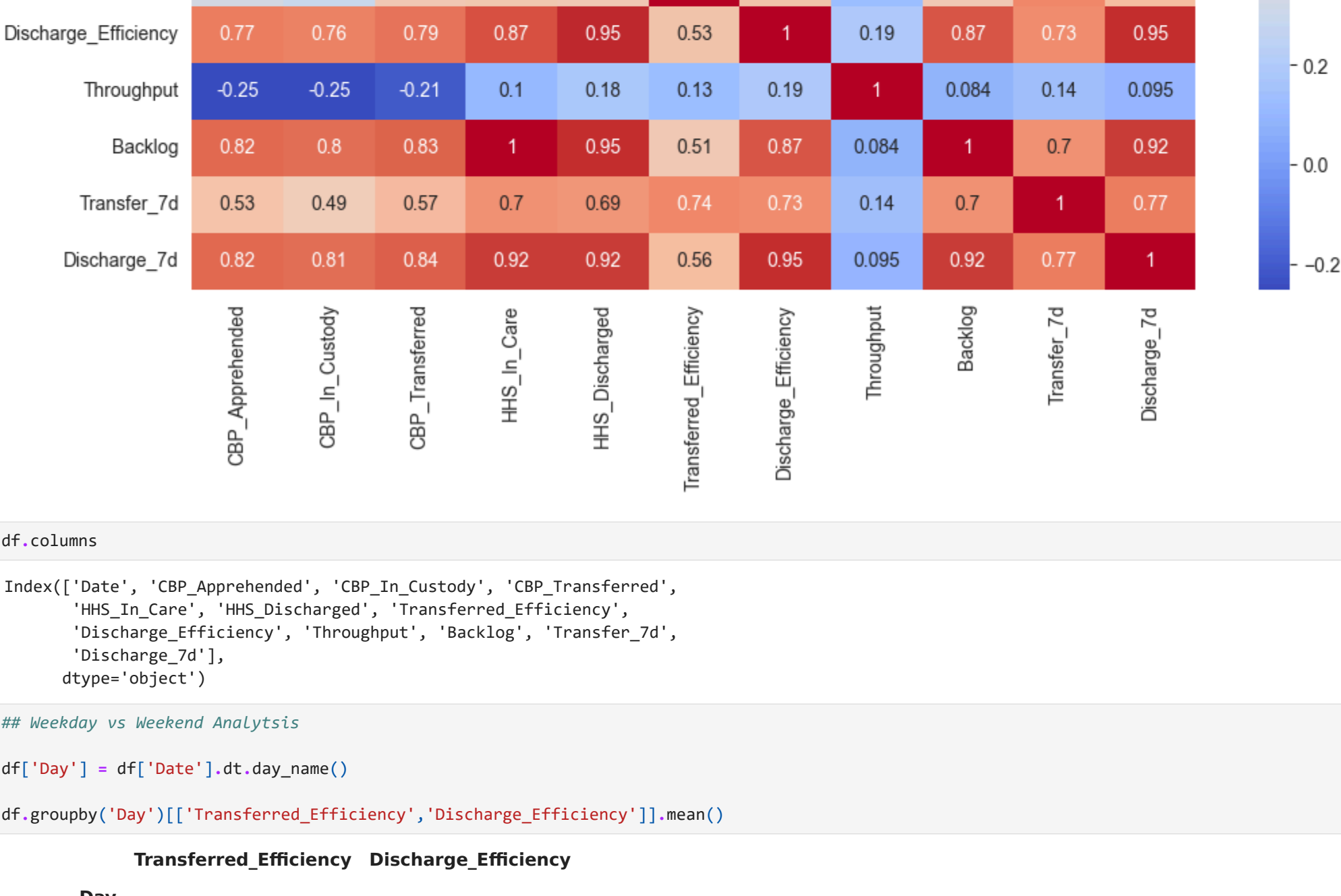
Out[42]:
```

Date	Backlog
651 2023-05-02	8735.0
650 2023-05-03	8790.0
649 2023-05-04	8711.0
584 2023-08-08	8750.0
583 2023-08-09	8776.0
...	...
444 2024-03-06	9095.0
443 2024-03-07	9021.0
441 2024-03-11	8840.0
440 2024-03-12	8924.0
439 2024-03-13	8938.0

117 rows x 2 columns

```
In [43]: # Correlation Heatmap

plt.figure(figsize=(12,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



```
In [44]: df.columns

Out[44]: Index(['Date', 'CBP_Apprehended', 'CBP_In_Custody', 'CBP_Transferred', 'HHS_In_Care', 'HHS_Discharged', 'Transferred_Efficiency', 'Discharge_Efficiency', 'Throughput', 'Backlog', 'Transfer_7d', 'Discharge_7d'], dtype='object')

In [45]: ## Weekday vs Weekend Analysis

df['Day'] = df['Date'].dt.day_name()

df.groupby('Day')[['Transferred_Efficiency', 'Discharge_Efficiency']].mean()

Out[45]:
```

Day	Transferred_Efficiency	Discharge_Efficiency
Friday	0.711818	0.024037
Monday	0.660516	0.021711
Sunday	0.631053	0.010737
Thursday	0.732629	0.027853
Tuesday	0.670485	0.018705
Wednesday	0.692006	0.022596

Key Insights

- 1. Transfer Efficiency fluctuates → possible CBP delays
- 2. Backlog increases when inflow > discharge
- 3. Discharge Efficiency relatively stable but dips during spikes

4. Bottlenecks occur during high intake periods

5. Weekend processing slower (if observed)

In []:

